

Manual tecnico por funcionalidade

Zeladoria Coopatos

Dependencias, arquivos, funcoes, rotas, fluxo e como explicar cada parte do codigo.

Como usar este PDF

Cada pagina explica uma funcionalidade do projeto. Use a tabela para responder onde esta no codigo, qual dependencia foi usada e como o fluxo funciona. As linhas podem mudar com commits, entao cite principalmente arquivo + funcao/rota/modelo.

Quando perguntarem	Como responder
Onde esta no codigo?	Cite arquivo e funcao/rota. Ex.: mapa fica em src/components/DashboardMaps.tsx no componente DashboardMaps.
Qual dependencia usou?	Cite a biblioteca: Leaflet para mapa, Socket.IO para tempo real, Prisma para banco, Cloudinary para midias.
Como funciona?	Explique o fluxo: frontend chama API, backend valida/salva com Prisma, Socket.IO atualiza em tempo real quando necessario.

Mapa rapido do projeto

Area	Arquivos principais
Rotas da aplicacao	src/App.tsx
Login funcionario	src/pages/EmployeeLogin.tsx + POST /employee-login em backend/src/routes/index.ts
Login admin	src/pages/AdminLogin.tsx + POST /admin-login em backend/src/routes/index.ts
Painel funcionario	src/pages/EmployeePanel.tsx
Dashboard admin	src/pages/Dashboard.tsx
Mapa	src/components/DashboardMaps.tsx
API backend	backend/src/server.ts + backend/src/routes/index.ts
Banco	prisma/schema.prisma
Whitelabel	src/config/brand.ts + BrandLogo.tsx + backend/src/config/brand.ts

1. Mapa de chamados e mapa de calor

O mapa mostra os chamados filtrados no painel administrativo. Ele tem modo marcadores e modo mapa de calor. Ao clicar em um pin, o Dashboard abre o detalhe do chamado.

Parte	O que falar / onde mostrar
Dependencias	leaflet, leaflet.markercluster, leaflet.heat e OpenStreetMap.
Arquivos principais	src/components/DashboardMaps.tsx; usado em src/pages/Dashboard.tsx na aba Mapa; dados vêm de GET /reports.
Funcoes, rotas ou modelos	DashboardsMaps; L.map(...).setView(...); L.markerClusterGroup(); L.heatLayer(); onSelectReport(report).

Fluxo da implementacao

1. Filtro admin Dashboard.tsx filtra chamados por categoria, status, funcionario e local.	2. Envia ao mapa Dashboard passa reports={filtered} para DashboardMaps.	3. Renderiza DashboardMaps cria L.map, camada OpenStreetMap, pins ou heatLayer.	4. Seleciona Clique no pin chama onSelectReport e abre detalhe no Dashboard.
---	---	---	--

Como explicar para o professor

“O mapa foi feito num componente separado chamado DashboardMaps. Ele recebe os chamados ja filtrados, pega latitude e longitude, cria um mapa Leaflet com OpenStreetMap e desenha pins ou heatmap. O clique no pin devolve o chamado para o Dashboard abrir o modal.”

Codigo-chave para apontar

src/components/DashboardMaps.tsx
<pre>const map = L.map(mapContainerRef.current).setView(center, 14) L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", ...).addTo(map) const markers = L.markerClusterGroup ? L.markerClusterGroup() : L.layerGroup() L.heatLayer(points, { radius: 30, blur: 20, maxZoom: 17 }).addTo(map)</pre>
src/pages/Dashboard.tsx
<pre><DashboardMaps reports={filtered} onSelectReport={report => setSelectedReport(report)} /></pre>

Perguntas provaveis

Pergunta	Resposta curta
Por que Leaflet?	Porque permite mapa web com OpenStreetMap, pins, cluster e heatmap.
De onde vem a localizacao?	Dos campos latitude e longitude do chamado, salvos no model Report.

2. Login, permissao e sessao

O sistema tem login de funcionario e admin. O admin so entra se estiver autenticado e ligado ao departamento administrativo. Tokens ativos impedem sessoes simultaneas.

Parte	O que falar / onde mostrar
Dependencias	react-router-dom, AuthContext, Express, Prisma, Socket.IO.
Arquivos principais	src/App.tsx; src/contexts/AuthContext.tsx; EmployeeLogin.tsx; AdminLogin.tsx; backend/src/routes/index.ts.
Funcoes, rotas ou modelos	ProtectedEmployee; ProtectedAdmin; loginEmployee; loginAdmin; logout; POST /employee-login; POST /admin-login; /employee-session/:id; /admin-session/:id.

Fluxo da implementacao

1. Login Tela envia credenciais para API.	2. Valida Backend consulta Employee/User com Prisma.	3. Token Backend grava activeSessionToken; frontend salva local/sessionStorage.	4. Protecao App.tsx libera /funcionario ou /dashboard.
---	--	---	--

Como explicar para o professor

“A autenticacao fica no AuthContext e as rotas protegidas no App.tsx. O backend valida o usuario, cria token ativo e, se houver outro login, emite force-logout por Socket.IO.”

Codigo-chave para apontar

src/App.tsx
ProtectedEmployee protege /funcionario ProtectedAdmin protege /dashboard e confere departamento administrativo
backend/src/routes/index.ts
router.post("/employee-login", async (req, res) => { ... }) router.post("/admin-login", async (req, res) => { ... })

Perguntas provaveis

Pergunta	Resposta curta
Como bloqueia admin comum?	App.tsx verifica departamento administrativo e backend valida sessao nas rotas /admin.
Como desloga outra aba?	Socket.IO envia force-logout quando muda o token ativo.

3. Criacao e gestao de chamados

Chamado e o centro do sistema. Funcionario cria chamado com categoria, prioridade, descricao, localizacao e midias. Admin cria, edita, exclui e atribui varios funcionarios.

Parte	O que falar / onde mostrar
Dependencias	React forms, Express, Prisma, Cloudinary e Socket.IO.
Arquivos principais	src/pages/EmployeePanel.tsx; src/pages/Dashboard.tsx; backend/src/routes/index.ts; prisma/schema.prisma.
Funcoes, rotas ou modelos	POST /reports; POST /admin/reports; PATCH /reports/:id; PATCH /admin/reports/:id; DELETE /admin/reports/:id; Report; ReportParticipant; ReportImage.

Fluxo da implementacao

1. Dados Frontend coleta titulo, categoria, prioridade, endereco/GPS e midias.	2. Midia Arquivos vao para Cloudinary e voltam como URL.	3. Salva Backend cria Report, imagens e participantes via Prisma.	4. Atualiza Emite reports-updated/report-created via Socket.IO.
--	--	---	---

Como explicar para o professor

“O chamado e salvo no model Report. A tela do funcionario usa EmployeePanel e o admin usa Dashboard. Antes de salvar, midias vao para Cloudinary. Depois o backend grava com Prisma e atualiza as telas por Socket.IO.”

Codigo-chave para apontar

backend/src/routes/index.ts
<pre>router.post("/reports", async (req, res) => { prisma.report.create(...) }) router.post("/admin/reports", async (req, res) => { ... })</pre>
prisma/schema.prisma
<pre>model Report { title, priority, latitude, longitude, images, messages, participants }</pre>

Perguntas provaveis

Pergunta	Resposta curta
Onde fica prioridade?	Campo priority do model Report e controles em EmployeePanel/Dashboard.
Como atribui funcionario?	Tabela ReportParticipant e rota /reports/:id/participants.

4. Upload de imagens, videos e audios

Arquivos podem aparecer em chamados, chats, anotacoes e perfil. O banco guarda URL/publicId, nao o arquivo bruto.

Parte	O que falar / onde mostrar
Dependencias	Cloudinary no frontend e SDK cloudinary no backend para excluir recursos.
Arquivos principais	EmployeePanel.tsx; Dashboard.tsx; backend/src/routes/index.ts.
Funcoes, rotas ou modelos	uploadImageToCloudinary; uploadReportMediaToCloudinary; cloudinary.config; cloudinary.uploader.destroy; POST /reports/:id/images; DELETE /report-images/:id.

Fluxo da implementacao

1. Seleciona Input file pega imagem, video ou audio.	2. Envia Frontend faz POST para Cloudinary.	3. Recebe Cloudinary devolve secure_url, public_id e resource_type.	4. Salva Backend/Prisma salva referencia; exclusao remove no Cloudinary.
--	---	---	--

Como explicar para o professor

"A midia fica no Cloudinary. No banco salvamos URL, publicId e tipo. Isso deixa o banco leve e permite excluir a midia usando publicId."

Codigo-chave para apontar

src/pages/EmployeePanel.tsx
uploadImageToCloudinary(file) -> FormData -> fetch Cloudinary
backend/src/routes/index.ts
cloudinary.config({ cloud_name, api_key, api_secret }) await cloudinary.uploader.destroy(publicId, { resource_type })

Perguntas provaveis

Pergunta	Resposta curta
Por que nao salvar no banco?	Para o banco nao ficar pesado; ele guarda so a referencia.
Como apaga?	Pelo publicId salvo no banco.

5. Chat do chamado

Cada chamado tem uma conversa com texto, emoji, midia, resposta, digitacao e check de leitura.

Parte	O que falar / onde mostrar
Dependencias	Socket.IO, Prisma, Cloudinary, emoji-picker-react, framer-motion.
Arquivos principais	EmployeePanel.tsx; backend/src/server.ts; backend/src/routes/index.ts; prisma/schema.prisma.
Funcoes, rotas ou modelos	sendMessage; markMessagesAsRead; join-report; new-message; POST /reports/:id/messages; POST /reports/:id/mark-read; ReportMessage; ReportMessageRead.

Fluxo da implementacao

1. Sala Ao abrir conversa, frontend emite join-report.	2. Mensagem sendMessage chama POST /reports/:id/messages.	3. Backend Salva ReportMessage e emite new-message.	4. Leitura mark-read grava ReportMessageRead e emite messages-read.
--	---	---	---

Como explicar para o professor

"O chat do chamado usa sala Socket.IO report-ID. Quando alguém envia mensagem, backend salva no Prisma e emite new-message para a sala. A leitura/check azul usa ReportMessageRead."

Codigo-chave para apontar

src/pages/EmployeePanel.tsx
<pre>socket.emit("join-report", selectedReport.id) socket.on("new-message", (message) => ...) const sendMessage = async () => fetch(/reports/\${id}/messages)</pre>
backend/src/routes/index.ts
<pre>router.post("/reports/:id/messages", async (...) => { const newMessage = await prisma.reportMessage.create(...) io.to(report-\${reportId}).emit("new-message", newMessage) })</pre>

Perguntas provaveis

Pergunta	Resposta curta
Como chega em tempo real?	Socket.IO emite new-message para a sala report-ID.
Como check azul funciona?	ReportMessageRead guarda ultima mensagem lida.

6. Chat privado

Conversa direta entre funcionarios, com texto, midia, emoji, resposta, edicao, exclusao, selecao multipla, digitacao e check azul.

Parte	O que falar / onde mostrar
Dependencias	Socket.IO, Prisma, Cloudinary, emoji-picker-react, framer-motion.
Arquivos principais	EmployeePanel.tsx; backend/src/routes/index.ts; backend/src/server.ts; prisma/schema.prisma.
Funcoes, rotas ou modelos	openPrivateConversation; sendPrivateMessage; markPrivateMessagesAsRead; join-private-conversation; private-message; private-messages-read; PrivateConversation; PrivateMessageRead.

Fluxo da implementacao

1. Abre POST /private-conversations cria ou acha conversa.	2. Sala Socket entra em private-conversation-ID.	3. Salva Backend salva PrivateMessage e midias.	4. Entrega Emite private-message e mark-read atualiza leitura.
---	--	---	--

Como explicar para o professor

“O privado funciona parecido com o chat do chamado, mas a sala e por conversa privada. O backend garante participantes, salva a mensagem e envia private-message. A leitura usa PrivateMessageRead.”

Codigo-chave para apontar

backend/src/routes/index.ts
<pre>router.post("/private-conversations", async (...) => { ... }) router.post("/private-conversations/:conversationId/messages", async (...) => { ... }) router.post("/private-conversations/:conversationId/mark-read", async (...) => { ... })</pre>
prisma/schema.prisma
<pre>model PrivateConversation { participants, messages } model PrivateMessageRead { conversationId, employeeId, lastReadMessageId }</pre>

Perguntas provaveis

Pergunta	Resposta curta
Como evita duplicar conversa?	A rota procura conversa existente antes de criar.
Como notifica privado?	createNotification cria PRIVATE_MESSAGE ou PRIVATE_MENTION.

7. Notificacoes, mencoes e contador

Notificacoes avisam mencoes com @, mensagens privadas, atribuicoes e atualizacoes. Podem ser lidas, apagadas e usadas para navegar ate o item.

Parte	O que falar / onde mostrar
Dependencias	Socket.IO, Prisma e React state.
Arquivos principais	backend/src/routes/index.ts; src/pages/EmployeePanel.tsx; prisma/schema.prisma.
Funcoes, rotas ou modelos	createNotification; GET /notifications/:employeeId; POST /notifications/:id/read; DELETE /notifications/:id; notification-created; report-message-notification; Notification.

Fluxo da implementacao

1. Evento Mensagem, mencao ou atribuicao dispara regra.	2. Registro createNotification grava Notification.	3. Socket Backend envia notification-created para employee-ID.	4. Tela EmployeePanel atualiza contador e painel.
---	--	--	---

Como explicar para o professor

“A notificacao fica salva no banco. Quando ocorre uma mencao ou mensagem, o backend chama createNotification e envia por Socket.IO para o funcionario certo.”

Codigo-chave para apontar

backend/src/routes/index.ts
<pre>const createNotification = async ({ recipientId, type, title, body }) => { const notification = await prisma.notification.create(...) io.to(employee-\${recipientId}).emit("notification-created", notification) }</pre>

Perguntas provaveis

Pergunta	Resposta curta
Onde fica @?	Mensagem envia mentionedEmployeeIds e backend cria MENTION/PRIVATE_MENTION.
Por que privado nao cria varias notificacoes?	PRIVATE_MESSAGE pode ser consolidada por conversa nao lida.

8. Anotacoes/observacoes no chamado

Anotacoes registram o andamento formal do chamado, separadas do chat. Podem ter texto e midia.

Parte	O que falar / onde mostrar
Dependencias	Prisma, Cloudinary e Express.
Arquivos principais	EmployeePanel.tsx; backend/src/routes/index.ts; prisma/schema.prisma.
Funcoes, rotas ou modelos	GET /reports/:id/notes; POST /reports/:id/notes; DELETE /reports/:reportId/notes/:noteId; ReportNote; ReportNoteMedia.

Fluxo da implementacao

1. Texto EmployeePanel coleta anotacao e arquivos.	2. Midia Arquivos opcionais vao para Cloudinary.	3. Salva Backend cria ReportNote e ReportNoteMedia.	4. Historico Tela recarrega notas e emite reports-updated.
--	--	---	--

Como explicar para o professor

“Anotacao e um registro operacional do que esta sendo feito no chamado. O backend salva em ReportNote e as midias em ReportNoteMedia.”

Codigo-chave para apontar

backend/src/routes/index.ts
<pre>router.post("/reports/:id/notes", async (...) => { const note = await prisma.reportNote.create({ data: ... }) })</pre>

Perguntas provaveis

Pergunta	Resposta curta
E diferente do chat?	Sim. Chat e conversa; anotacao e registro/historico do andamento.
Onde aparece?	Nos detalhes do chamado no EmployeePanel.

9. Dashboard: indicadores e graficos

Indicadores transformam chamados em dados de gestao: status, prioridade, produtividade, departamentos, evolucao mensal e locais recorrentes.

Parte	O que falar / onde mostrar
Dependencias	Recharts e React useMemo.
Arquivos principais	src/pages/Dashboard.tsx.
Funcoes, rotas ou modelos	analytics = useMemo(...); analyticsFiltered; exportAnalyticsPdf; cards clicaveis para abrir chamados filtrados.

Fluxo da implementacao

1. Busca Dashboard carrega GET /reports.	2. Filtra analyticsFiltered usa filtros proprios.	3. Calcula useMemo monta arrays para Recharts.	4. Exibe Cards/graficos e exportacao PDF.
--	---	--	---

Como explicar para o professor

“Os graficos sao calculados no frontend a partir dos chamados. O Dashboard filtra os reports, calcula os agregados com useMemo e o Recharts desenha.”

Codigo-chave para apontar

src/pages/Dashboard.tsx
<pre>const analytics = useMemo(() => { const sourceReports = analyticsFiltered; const statusData = statuses.map(...); return { statusData, monthlyData, ... }; }, [...])</pre>

Perguntas provaveis

Pergunta	Resposta curta
Qual biblioteca?	Recharts.
Os filtros dos graficos mexem nos chamados?	Sao estados separados: analytics* para indicadores e filter* para chamados.

10. Admin: funcionarios, usuarios e departamentos

Admin gerencia funcionarios, acessos e departamentos. Funcionario tem foto, nascimento, CPF, email, validacao por email, departamento e status.

Parte	O que falar / onde mostrar
Dependencias	React forms/modais, Express, Prisma, Nodemailer, Cloudinary e auditoria.
Arquivos principais	Dashboard.tsx; backend/src/routes/index.ts; prisma/schema.prisma.
Funcoes, rotas ou modelos	/admin/employees; /admin/users; /admin/departments; sendEmployeeVerificationEmailInBackground; createAuditLog.

Fluxo da implementacao

1. Modal Dashboard coleta dados e validacoes.	2. API Rotas /admin salvam Employee/User/Department.	3. Extras Envia validacao e registra AuditLog.	4. Lista Dashboard atualiza tabelas e filtros.
---	--	--	--

Como explicar para o professor

"O CRUD admin fica no Dashboard e nas rotas /admin. Cada alteracao importante gera auditoria e, quando mexe no email, dispara validacao por Nodemailer."

Codigo-chave para apontar

backend/src/routes/index.ts
<pre>router.post("/admin/employees", async (...) => { ... }) router.patch("/admin/employees/:id", async (...) => { ... }) router.post("/admin/departments", async (...) => { ... })</pre>

Perguntas provaveis

Pergunta	Resposta curta
Como valida email?	Backend cria token e envia link via Nodemailer.
Como audita?	createAuditLog grava entidade, acao, ator e metadata.

11. Auditoria e exportacoes

Auditoria registra quem fez alteracoes. Exportacoes geram CSV/PDF de chamados, funcionarios, auditoria e indicadores.

Parte	O que falar / onde mostrar
Dependencias	Prisma AuditLog, frontend HTML/print para PDF e Express para CSV.
Arquivos principais	Dashboard.tsx; backend/src/routes/index.ts; prisma/schema.prisma.
Funcoes, rotas ou modelos	createAuditLog; GET /admin/audit-logs; GET /admin/exports/:type; exportAdminCsv; exportFilteredReportsPdf; exportEmployeesPdf; exportAuditPdf; exportAnalyticsPdf.

Fluxo da implementacao

1. Acao Admin cria/edita/desativa algo.	2. Log Backend chama createAuditLog.	3. Consulta Dashboard chama /admin/audit-logs.	4. Exporta CSV/PDF usa filtros e colunas escolhidas.
---	--	--	--

Como explicar para o professor

“Auditoria e o model AuditLog. As rotas admin chamam createAuditLog. O Dashboard visualiza e exporta relatorios.”

Codigo-chave para apontar

backend/src/routes/index.ts
<pre>const createAuditLog = async ({ entity, entityId, action, summary, metadata }) => { await prisma.auditLog.create({ data: ... }) }</pre>

Perguntas provaveis

Pergunta	Resposta curta
Quem aparece como responsavel?	O backend registra actorId/actorName pela sessao admin.
Como exporta PDF?	Dashboard monta HTML formatado e usa print/save as PDF.

12. Whitelabel e personalizacao

Permite trocar nome, logo, slogan e cores para usar o sistema como modelo para outra empresa.

Parte	O que falar / onde mostrar
Dependencias	React, localStorage, CSS variables e BrandLogo.
Arquivos principais	src/config/brand.ts; src/components/BrandLogo.tsx; backend/src/config/brand.ts; Dashboard.tsx; src/main.tsx.
Funcoes, rotas ou modelos	applyBrandPreset; saveBrandPreset; restoreBrandPreset; useBranding; BrandLogo; persistBrandPreset.

Fluxo da implementacao

1. Preset defaultBrandPreset define nome, logo e cores.	2. CSS applyBrandPreset grava variaveis no HTML.	3. Admin Dashboard salva preset no localStorage.	4. Uso BrandLogo e telas leem useBranding.
---	--	--	--

Como explicar para o professor

"Whitelabel foi feito com preset central. Componentes nao usam logo/cor fixa; leem brandPreset. As cores viram variaveis CSS, entao mudar o preset atualiza o site."

Codigo-chave para apontar

src/config/brand.ts
<pre>export const defaultBrandPreset = { appName, shortName, logoSrc, colors } export const applyBrandPreset = (preset) => root.style.setProperty("--primary", preset.colors.primary)</pre>
src/main.tsx
<pre>applyBrandPreset(getStoredBrandPreset())</pre>

Perguntas provaveis

Pergunta	Resposta curta
E multiempresa real?	Hoje salva no navegador; para multiempresa real salvaria por tenant/dominio no banco.
Onde troca logo?	src/config/brand.ts ou Configuracoes > Personalizacao.

13. Emails: validacao e recuperacao

Envia email de validacao de cadastro e recuperacao de matricula.

Parte	O que falar / onde mostrar
Dependencias	Nodemailer com SMTP e Prisma para token/validacao.
Arquivos principais	backend/src/routes/index.ts; backend/src/config/brand.ts; VerifyEmployeeEmail.tsx; EmployeeLogin.tsx.
Funcoes, rotas ou modelos	mailTransporter; sendEmployeeVerificationEmail; POST /employee/recover-registration; GET /employee/verify-email/:token.

Fluxo da implementacao

1. Cadastro Admin cria ou muda email.	2. Token Backend salva emailVerificationToken.	3. Email Nodemailer envia link para /validar-email/:token.	4. Valida Frontend chama API; backend marca emailVerifiedAt.
---	--	--	--

Como explicar para o professor

“Email e enviado pelo backend com Nodemailer. O link abre uma tela do frontend e essa tela chama a API para validar o token.”

Codigo-chave para apontar

backend/src/routes/index.ts
<pre>const mailTransporter = nodemailer.createTransport({...}) await mailTransporter.sendMail({ to, subject, html }) router.get("/employee/verify-email/:token", async (...) => { ... })</pre>

Perguntas provaveis

Pergunta	Resposta curta
Por que link vai para frontend?	Para mostrar tela amigavel antes/depois da API validar.
Como evita reusar?	Ao validar, token e limpo/inutilizado.

14. Banco de dados e Prisma

Prisma modela tabelas e relacionamentos e gera o client usado no backend.

Parte	O que falar / onde mostrar
Dependencias	Prisma, @prisma/client, pg, PostgreSQL/Neon.
Arquivos principais	prisma/schema.prisma; prisma/migrations; backend/src/prisma/client.ts; backend/src/routes/index.ts.
Funcoes, rotas ou modelos	prisma.report.findMany/create/update; prisma.employee; prisma.notification; prisma.auditLog; migrations.

Fluxo da implementacao

1. Schema schema.prisma define modelos.	2. Migration prisma/migrations versiona alteracoes.	3. API Rotas usam Prisma Client.	4. JSON Frontend recebe dados pela API.
---	---	--	---

Como explicar para o professor

“O schema.prisma define a estrutura do banco. O backend nao escreve SQL manual em cada tela; usa Prisma Client para consultar e salvar dados.”

Codigo-chave para apontar

prisma/schema.prisma
<pre>model Report { ... messages ReportMessage[] } model Employee { ... reports Report[] } model Notification { recipientId Int type String readAt DateTime? }</pre>
backend/src/routes/index.ts
<pre>const reports = await prisma.report.findMany({ include: reportInclude })</pre>

Perguntas provaveis

Pergunta	Resposta curta
Onde vejo tabelas?	prisma/schema.prisma.
Onde vejo alteracoes?	prisma/migrations.

15. Deploy e variaveis de ambiente

Frontend na Vercel, backend no Render e banco no Neon. Segredos ficam em variaveis de ambiente.

Parte	O que falar / onde mostrar
Dependencias	Vercel, Render, Neon, Cloudinary e SMTP.
Arquivos principais	vercel.json; package.json; backend/src/server.ts; README.md; .env local.
Funcoes, rotas ou modelos	VITE_API_URL; DATABASE_URL; CLOUDINARY_*; MAIL_*; PUBLIC_APP_URL; PORT.

Fluxo da implementacao

1. Frontend Vercel faz build Vite e serve SPA.	2. Backend Render roda Express na porta do ambiente.	3. Banco Render usa DATABASE_URL do Neon.	4. Servicos Cloudinary/SMTP usam chaves de ambiente.
--	--	---	--

Como explicar para o professor

“A estrutura separa responsabilidades: Vercel entrega React, Render roda API e Neon guarda dados. O frontend chama a API pela VITE_API_URL.”

Codigo-chave para apontar

backend/src/server.ts
<pre>const PORT = Number(process.env.PORT) 3333 server.listen(PORT, ...)</pre>
README.md
<pre>VITE_API_URL=... DATABASE_URL=... CLOUDINARY_CLOUD_NAME=... MAIL_USER=...</pre>

Perguntas provaveis

Pergunta	Resposta curta
Por que VITE_API_URL?	Para local usar localhost e producao usar API do Render.
Onde fica rewrite SPA?	vercel.json.

Resumo das tecnologias para slide

Tecnologia	Uso
React + TypeScript + Vite	Interface web e build do frontend.
Tailwind + shadcn/ui + Radix UI	UI, modais, inputs, selects e toasts.
Node.js + Express	API backend e regras.
Prisma + PostgreSQL/Neon	Banco, modelos e migrations.
Socket.IO	Chats, notificacoes, presenca, leitura e logout em tempo real.
Cloudinary	Upload de imagens, videos e audios.
Nodemailer	Emails de validacao e recuperacao.
Leaflet + OpenStreetMap	Mapa com pins, cluster e heatmap.
Recharts	Graficos e indicadores.
Vercel + Render	Deploy frontend/backend.

Cola rapida: perguntas comuns

Pergunta	Resposta curta
Como foi feito o mapa?	Leaflet em DashboardMaps.tsx; recebe chamados filtrados, usa latitude/longitude, cria pins e heatLayer.
Como o chat funciona?	EmployeePanel envia para API; backend salva com Prisma e emite Socket.IO para sala do chamado/conversa.
Onde estao as rotas?	backend/src/routes/index.ts. Socket.IO base em backend/src/server.ts.
Onde estao as tabelas?	prisma/schema.prisma.
Como salva midia?	Cloudinary armazena arquivo; banco salva URL/publicId/resourceType.
Como funciona notificacao?	createNotification salva Notification e emite notification-created para employee-ID.
Como funciona check azul?	mark-read atualiza ReportMessageRead ou PrivateMessageRead e emite evento de leitura.
Como funciona admin?	Dashboard.tsx chama rotas /admin; backend valida sessao e departamento administrativo.
Como funciona whitelabel?	src/config/brand.ts aplica CSS variables e BrandLogo usa logo configurada.
Como funciona auditoria?	Rotas admin chamam createAuditLog, que grava AuditLog com ator, entidade e resumo.